Universidade Federal de São Paulo
Instituto de Ciência e Tecnologia
Arquitetura e Organização de Computadores - Turma IB

Sistema de recomendação turística em assembly MIPS com foco em acessibilidade

Integrantes:
Caio Peternela de Souza - 156317
João Vitor Mâncio Chaves - 176534
José Luiz de Oliveira Ferreira - 176539
Natália Vitória Amaral do Val - 176626
Docente responsável: Profa. Dra. Thaína A. A. Tosta

São José dos Campos
2025/2

Universidade Federal de São Paulo
Instituto de Ciência e Tecnologia
Arquitetura e Organização de Computadores - Turma IB

Sistema de recomendação turística em assembly MIPS com foco em acessibilidade

Segundo relatório do projeto de Arquitetura e Organização de Computadores (Turma IB) da Universidade Federal de São Paulo para cumprimento dos requisitos de avaliação e aprovação na disciplina.

São José dos Campos
2025/2

Resumo

O projeto tem como objetivo desenvolver um sistema que oferece experiências de turismo personalizadas, acessíveis e inclusivas. Por meio de um *quiz* interativo, o programa sugere destinos adequados ao perfil do usuário, considerando preferências individuais, da família e limitações econômicas. Desse modo, oferece opções variadas como parques, museus e centros históricos, além de propor ambientes adaptados que atendem às necessidades de pessoas neurodivergentes, garantindo experiências mais calmas e seguras. Desenvolvido em linguagem *assembly MIPS* e executado no simulador *MARS*, o sistema integra uma interface *web* para interação amigável e um *middleware* responsável por conectar o navegador ao processamento no simulador. Assim, o projeto vai além do desenvolvimento técnico, representando uma integração entre tecnologia, empatia e cidadania, ao demonstrar como soluções digitais contribuem para a construção de experiências turísticas mais humanas, seguras e igualitárias.

1 Introdução

No Brasil, uma parcela significativa da população ainda enfrenta barreiras para realizar viagens de turismo. Segundo pesquisa encomendada pelo Ministério do Turismo [1], 44,4% dos brasileiros nunca viajaram a turismo pelo país. Além das limitações econômicas, muitas famílias enfrentam dificuldades em encontrar destinos que agradem a todos os seus integrantes, seja por diferenças de idade, interesses ou preferências de lazer. Uma pesquisa do IBGE [2] revelou que a falta de dinheiro é o principal motivo para não viajar, citado por 48,9% dos domicílios que não realizaram viagens. Esses dados evidenciam que, para promover o turismo de forma inclusiva e acessível, é essencial considerar as diversas realidades econômicas e as preferências variadas das famílias brasileiras. Incentivar o turismo acessível e criar plataformas que auxiliem no planejamento de viagens personalizadas são passos importantes para superar esses desafios.

O projeto tem como proposta principal oferecer uma experiência de turismo personalizada e acessível, na região sul do Brasil, que integra tecnologia e lazer. Por meio de um *quiz* interativo, o sistema recomenda destinos de viagem com base nas preferências e características do usuário, incentivando o turismo consciente e a valorização da diversidade cultural e ambiental. A partir das respostas, o visitante recebe sugestões de cidades e locais que melhor se adequam ao seu perfil, como parques, museus, restaurantes ou centros históricos, promovendo o contato com diferentes realidades e estilos de vida. Nessa perspectiva, o projeto também buscou, desde a concepção, ampliar a inclusão social, considerando as necessidades de grupos que frequentemente enfrentam barreiras de acessibilidade no turismo tradicional. Entre eles, destacam-se pessoas neurodivergentes, que podem se beneficiar de ambientes adaptados e experiências mais calmas e seguras dependendo de suas seletividades. Segundo Gomes (2008), as anormalidades sensório-perceptuais afetam cerca de 90% dos autistas; enquanto alguns apresentam hipersensibilidade a ruídos, outros podem reagir positivamente a estímulos sonoros. Considerando essa heterogeneidade, o sistema proposto atua não apenas como um guia turístico inteligente, mas também como uma ferramenta para tornar a experiência turística mais acolhedora, segura e, principalmente, acessível [3].

O turismo ecológico e a valorização da natureza têm se destacado como importantes atividades de lazer e educação ambiental. Em "5 cidades em Minas para um roteiro botânico incrível nesta primavera" [4] foram apresentadas opções de visitação a parques e jardins botânicos em Minas Gerais, incentivando o contato com a biodiversidade local. Contudo, a experiência de visitação nem sempre é plenamente acessível para todos os públicos, especialmente crianças neurodivergentes, que demandam atenção diferenciada

e ambientes adaptados. Segundo Caetano e Pereira (2025) [5],a inclusão de pessoas com Transtorno do Espectro Autista (TEA) em espaços culturais, como museus e centros de ciências, ainda enfrenta desafios relacionados à acessibilidade física, sensorial e comunicacional. Os autores destacam a importância de estratégias que considerem a diversidade cognitiva e sensorial dos visitantes, promovendo experiências que reduzam a sobrecarga e favoreçam a interação significativa com o ambiente. Essa perspectiva pode ser estendida aos parques botânicos e áreas naturais, que também necessitam de mecanismos adaptativos — como horários especiais, rotas tranquilas e mediação adequada — para acolher plenamente crianças neurodivergentes. Por conta disso, o sistema de recomendação demandou uma base de dados de locais que demonstrassem um compromisso institucional e legal com a acessibilidade sensorial. Após extensa pesquisa, a escolha recaiu sobre a Região Sul do Brasil, abrangendo os estados do Rio Grande do Sul, Paraná e Santa Catarina, por apresentarem um arcabouço legislativo mais avançado e iniciativas de governo que incentivam a inclusão de pessoas com TEA em ambientes públicos e turísticos [6]. No podcast Introvertendo, um episódio intitulado "Autista da Região Sul"relata as experiências de pessoas com autismo vivendo no sul do país. Os entrevistados afirmam que, embora ainda haja muito a melhorar, essa região já possui mais investimento e locais melhores adaptados para pessoas autistas.[7]. Um exemplo disso é o Programa TEAcolhe (RS), cujo objetivo é implementar a Lei Estadual n° 15.322/2019, que instituiu a Política de Atendimento Integrado à Pessoa com Transtorno do Espectro Autisma no âmbito do Rio Grande do Sul. Esta lei é destinada a garantir e a promover o atendimento às necessidades específicas das pessoas com autismo, visando ao desenvolvimento pessoal, à inclusão social, à cidadania e ao apoio às suas famílias.[8]

Tais regulamentações facilitam a identificação e catalogação de atrativos turísticos que já possuem protocolos de atendimento diferenciado, dependendo das necessidades de um baixo nível de estímulo sonoro ou visual. Além de possuir fatores cruciais para a segurança e o conforto do público neurodivergente.

É precisamente para mitigar esses desafios sensoriais que o diferencial deste projeto se manifesta. A perspectiva de *Temple Grandin* [9], cientista e autora no espectro autista, corrobora a urgência dessa abordagem. Ela descreve como, para muitos indivíduos autistas, ambientes ruidosos podem tornar a comunicação incompreensível, dada a dificuldade em isolar a fala humana do ruído de fundo. O projeto atuou diretamente sobre essa barreira ao permitir que os usuários filtrassem destinos. Tais destinos já haviam sido estudados anteriormente para garantir sua acessibilidade. Isto significa que todas as informações de qualquer local que escolhido pelo *quiz* foram

verificadas para que entrassem nessa seleção. Essa filtragem, com base no nível de tolerância a estímulos do usuário, transformou a viagem que poderia ser para ambientes potencialmente hostis em espaços acolhedores e viáveis para o lazer.

Além da dimensão sensorial, o projeto aborda a barreira do estigma social, que frequentemente isola pessoas autistas. *Grandin* também aponta que a percepção pública sobre o autismo costuma ser limitada a um estereótipo infantil e severo, o que acaba por invisibilizar a existência e as necessidades de adolescentes e adultos no espectro. Nesse sentido, o diferencial da ferramenta proposta será também promover ativamente a inclusão. Ao facilitar o planejamento de atividades para indivíduos de todas as idades, o projeto contribui para a visibilidade e a participação de jovens e adultos autistas na vida cultural, reforçando seu direito ao lazer.

Nesse contexto, a tecnologia surge como uma ferramenta facilitadora. Fernandes e *Bernier* (2021) [10] argumentam que, nos destinos turísticos inteligentes, a acessibilidade virtual — ou seja, as condições de acesso às informações nos portais oficiais — é um elemento fundamental. Sistemas que fornecem informações acessíveis são cruciais para conferir autonomia aos visitantes, auxiliando-os na tomada de decisão e na forma como interagem com o destino, o que pode incluir funcionalidades como a prévia visualização do ambiente e o agendamento em horários de menor fluxo, reduzindo assim a ansiedade de pessoas com necessidades sensoriais específicas.

Com base nesses princípios, o sistema proposto uniu tecnologia e inclusão social por meio de um programa desenvolvido em linguagem *assembly MIPS* e executado no simulador *MARS*, além de uma interface visual para a realização do *quiz*.

O sistema possui um *quiz* interativo, no qual o usuário responde a perguntas sobre preferências e necessidades específicas. Com base nas respostas, o programa define um destino de visitação personalizado na região sul do país, sugerindo o espaço cultural mais adequado ao perfil do visitante, juntamente com o hotel e a sugestão de restaurante. Além disso, também foram disponibilizados locais na região que, mesmo não sendo o local vencedor, poderiam ser visitados, respeitando as condições de sensibilidade do usuário. Assim, o projeto amplia o conceito de turismo inteligente, tornando-se um exemplo de como soluções tecnológicas podem promover o bem-estar, a empatia e o acesso igualitário ao lazer e à cultura.

1.1 Objetivos

O objetivo geral deste trabalho é aplicar os conceitos fundamentais de arquitetura e organização de computadores no desenvolvimento de um sis-

tema interativo implementado em linguagem *assembly MIPS* e executado no simulador *MARS*, capaz de atuar como uma ferramenta de guia turístico personalizado que considera as diversas necessidades dos visitantes, com foco especial nas barreiras enfrentadas por pessoas com TEA. A aplicação gera roteiros personalizados (para locais como parques e restaurantes) a partir de um *quiz* interativo disponibilizado em uma interface de usuário Web, promovendo uma experiência de lazer mais acolhedora, em sintonia com os princípios do turismo inteligente e da inclusão social.

- Desenvolver um módulo de *quiz* interativo em *assembly MIPS* para coletar dados sobre as preferências de lazer e as necessidades específicas de acessibilidade do visitante;
- Implementar a lógica do sistema para processar as respostas do *quiz* e gerar um roteiro de visitação personalizado, sugerindo locais e atividades que melhor se adequem ao perfil do usuário e, também, atividades ao redor do destino vencedor, para que, assim, o usuário tenha mais opções de lazer que também respeitem suas seletividades;
- Desenvolver o *Front-end Web (index.html)* para atuar como a Interface de usuário gráfica, responsável por coletar de forma as entradas do quiz e substituir a interação de console *MIPS*;
- Configurar o *Back-end Assembly MIPS*, garantindo que execute a lógica central do sistema (cálculo de score e tomada de decisão);

2 Materiais e métodos

O código implementado apresenta um sistema de recomendação de destinos turísticos no Sul do Brasil, considerando o Perfil Sensorial (autismo) e seletividade alimentar do usuário. Ele coleta as seguintes entradas para calcular um *Score* para cada destino:

1. Estado (PR, SC, ou RS);
2. Orçamento (econômico ou conforto/luxo);
3. Atividade preferida (natureza, museu/cultura, parques/diversão, ou praia);
4. Perfil sensorial (calmo/silencioso ou agito/estímulos);
5. Seletividade alimentar (sim ou não).

Dessa forma, o destino com a maior pontuação dentro do estado selecionado é o vencedor.

Tabela 1: Locais Turísticos - Sul do Brasil

ID	**Nome do Local**	**Estado**	**Atividade**	**Sensorial**
0	Parque das Aves (Foz - PR)	1 (PR)	1 (Natureza)	1 (Calmo)
1	Vila Velha (Ponta Grossa - PR)	1 (PR)	1 (Natureza)	1 (Calmo)
2	Museu Oscar Niemeyer (Curitiba - PR)	1 (PR)	2 (Museu)	1 (Calmo)
3	Praia Mansa (Matinhos - PR)	1 (PR)	4 (Praia)	1 (Calmo)
4	Beto Carrero (Penha - SC)	2 (SC)	3 (Diversão)	2 (agito)
5	Unipraias (Balneário - SC)	2 (SC)	1 (Natureza)	1 (Calmo)
6	Zoo Pomerode (Pomerode - SC)	2 (SC)	1 (Natureza)	1 (Calmo)
7	Praia do Rosa (Imbituba - SC)	2 (SC)	4 (Praia)	1 (Calmo)
8	Gramado Zoo (Gramado - RS)	3 (RS)	1 (Natureza)	1 (Calmo)
9	Vila da Mônica (Gramado - RS)	3 (RS)	3 (Diversão)	2 (agito)
10	Geo Museu (Gramado - RS)	3 (RS)	2 (Museu)	1 (Calmo)
11	Praia da Guarita (Torres - RS)	3 (RS)	4 (Praia)	1 (Calmo)
12	Yup Star Roda Gigante (Foz - PR)	1 (PR)	3 (Diversão)	1 (Calmo)
13	Oceanic Aquarium (Balneário - SC)	2 (SC)	3 (Diversão)	1 (Calmo)
14	Mini Mundo (Gramado - RS)	3 (RS)	3 (Diversão)	1 (Calmo)
15	Jardim Botânico (Curitiba - PR)	1 (PR)	1 (Natureza)	1 (Calmo)
16	Snowland (Gramado - RS)	3 (RS)	3 (Diversão)	2 (agito)
17	Dreamland Museu de Cera (Foz - PR)	3 (PR)	2 (Museu)	1 (calmo)

Além disso, para cada destino, há sugestões baseadas no orçamento (econômico 'B' ou conforto/luxo 'C') e seletividade alimentar (simples 'S' ou complexo 'C').

Tabela 2: Hospedagem e Alimentação

ID	**Local**	**Hotel B**	**Hotel C**	**Rest. S**	**Rest. C**
0	Parque Aves	*Ibis* Foz	Recanto Cataratas	Barracão *Happy Hour*	Porto Canoas
4	Beto Carrero	Hotel Lorimar	Solar Pedra da Ilha	Julio Lanches	Petisqueira
8	Gramado Zoo	Hotel *Sky*	*Wyndham* termas	Sabor Nonna	*Hoppner*

Já o *Score* de um local é calculado através de pontos somados ou subtraídos no cálculo final, filtrando pelo estado escolhido:

- *Match* sensorial: se o perfil sensorial do local for igual ao do usuário, adiciona +40 pontos;

- Penalidade de segurança: se o usuário exige que o local seja calmo (1), mas o local é de agito (2), subtrai -100 pontos;
- *Match* atividade: se a atividade do local for igual à preferida do usuário, adiciona +30 pontos;
- Custo e alimentação: adiciona +15 pontos (peso custo) + 15 pontos (peso alimentação). O total são +30 pontos base para todos os locais, pois não dependem das escolhas do usuário no cálculo principal. Esta escolha é somente para o *Score* não retornar valores negativos.

O *Score* máximo é a somatória dos pesos $40 + 30 + 15 + 15 = 100$ pontos.

2.1 Base de Dados

A persistência de dados do sistema foi projetada para operar inteiramente na memória RAM simulada através da seção `.data` do MIPS. Diferente de linguagens de alto nível que encapsulam propriedades em objetos, a arquitetura deste projeto utiliza o conceito de vetores Paralelos para estruturar as informações dos 18 destinos turísticos mapeados. Esta abordagem permite que o processador acesse diferentes atributos de um mesmo local (como nome, estado, perfil sensorial e opções de logística) utilizando um único índice numérico (ID de 0 a 17) como chave de acesso, garantindo eficiência no ciclo de busca e comparação.

Para assegurar a integridade dos dados e o desempenho do barramento de memória, todas as estruturas vetoriais foram declaradas utilizando a diretiva de alinhamento `.align 2`. Esta diretiva força o armazenamento dos dados em endereços múltiplos de 4 bytes, alinhando-se às fronteiras de palavra (*word*) da arquitetura MIPS. Essa precaução é importante, pois tentativas de carregar palavras (`lw`) de endereços não alinhados resultariam em exceções de hardware (*address error exception*), interrompendo a execução do algoritmo.

Os locais armazenados no banco de dados são composta pelas strings de nomes dos pontos turísticos, hotéis e restaurantes, armazenadas como cadeias de caracteres ASCII terminadas em nulo (`.asciiz`). Para manipular essas strings, o sistema utiliza vetores de ponteiros, como o `listaLugares` e os vetores logísticos (`ptrHoteisB`, `ptrHoteisC`, etc.). Cada posição nestes vetores armazena o endereço de memória de 32 bits onde a string começa. Durante a execução, o algoritmo calcula o deslocamento de memória necessário multiplicando o ID do local por 4 (via instrução de deslocamento lógico `sll`), acessa o ponteiro correto e o carrega no registrador de argumento para as chamadas de sistema (*syscalls*) de impressão.

Paralelamente aos vetores de endereços, o sistema mantém vetores de atributos inteiros que sustentam a lógica de decisão do algoritmo de *Score*. O vetor `dadosEstado` armazena os identificadores geográficos (1 para PR, 2 para SC, 3 para RS), permitindo a filtragem imediata pelo usuário. Os vetores `listaAtividades` e `perfilSensorial` contêm os códigos numéricos que classificam cada local quanto ao tipo de lazer (Natureza, Museu, Diversão, Praia) e ao nível de estímulo (calmo ou agitado). É através da comparação aritmética direta entre os valores armazenados nestes vetores e as entradas do usuário que o sistema aplica as bonificações de pontuação ou as penalidades de segurança.

Por fim, a base de dados implementa uma estratégia de otimização de memória através da reutilização de ponteiros para a logística de serviços. Como múltiplos destinos turísticos podem estar situados na mesma cidade (compartilhando, portanto, a mesma infraestrutura hoteleira e gastronômica), o sistema evita a duplicidade de strings na memória. Por exemplo, os vetores de hotéis para os locais situados em Gramado (IDs 8, 14 e 16) apontam para os mesmos endereços de memória que contêm as strings dos hotéis daquela região.

2.2 Implementação

O sistema implementa um fluxo de execução sequencial que integra cálculo aritmético, estruturas de repetição e desvios condicionais para filtrar e ordenar as informações. As operações principais foram divididas nos seguintes módulos funcionais:

1. Coleta e Validação de Entradas;
2. Algoritmo de Recomendação e Score (com suporte a múltiplos vencedores);
3. Sugestão contextualizada de serviços (com filtro de duplicidade);
4. Funções de Exploração Adicional;
5. Persistência de Dados.

2.2.1 Coleta e Validação de Entradas

A sub-rotina *validaEntrada* é responsável por realizar a leitura segura de um valor inteiro através da *syscall li $v0, 5*, garantindo que a entrada fornecida pelo usuário respeite os limites mínimos e máximos definidos em $a1 e $a2. Caso o valor esteja fora desse intervalo, a rotina exibe uma

mensagem de erro (msgErro) e reinicia o processo de leitura (*j askInput*), assegurando que apenas entradas válidas sejam aceitas. Essa validação é fundamental para a integridade dos dados de entrada, pois valores fora do esperado comprometeriam o cálculo da recomendação e a precisão da fórmula de pontuação.

2.2.2 Algoritmo de Recomendação e *Score*

O núcleo de decisão é o *loopScore*, que itera sobre todos os 18 destinos da base de dados.

- **Filtro Rígido (*bne $t2, $s0, nextIter*)**: aplica um filtro inicial que desconsidera imediatamente qualquer destino cujo estado (*dadosEstado*) não corresponda ao estado escolhido pelo usuário ($s0).

- **Cálculo do Score**: implementa a fórmula de pontuação ponderada. A lógica inclui: *match* sensorial (+40 pontos), *match* atividade (+30 pontos), e uma penalidade de segurança crítica (-100 pontos) aplicada se o usuário busca um ambiente calmo ($s3=1) e o local é classificado como agito ($t3=2). Pesos base de custo e alimentação são somados (+30 pontos) para manter a escala positiva.

- **Tratamento de Empates (Buffer)**: diferente de algoritmos simples de máximo, o sistema utiliza um buffer (*bufferVencedores*) para armazenar múltiplos líderes. Se o score calculado for maior que o recorde (*bgt*), o buffer é limpo e o novo líder é salvo. Se o score for idêntico ao recorde (*beq*), o ID do local é anexado ao buffer, permitindo que o sistema recomende todas as melhores opções disponíveis.

2.2.3 Sugestão Contextualizada de Serviços

A lógica de logística adapta as sugestões ao perfil do usuário de forma a verificar a duplicidade. O sistema seleciona o hotel econômico (*ptrHoteisB*) ou conforto (*ptrHoteisC*) baseado no orçamento ($s1), e prioriza o restaurante simples (*ptrRestS*) caso haja seletividade alimentar ($s4=1). Além disso, como múltiplos locais vencedores podem pertencer à mesma cidade, o algoritmo verifica, antes da impressão, se o endereço de memória do hotel atual já foi referenciado por um vencedor anterior. Caso positivo, a impressão dos detalhes logísticos é suprimida e substituída por uma mensagem informativa ("Mesma logística sugerida acima"), evitando poluição visual na interface.

2.2.4 Funções de Exploração Adicional

Para enriquecer a experiência, o sistema implementa duas funcionalidades de descoberta:

- **Sugestão cruzada (bate-volta)**: O loop *loopBateVolta* realiza uma varredura no banco de dados buscando um local que compartilhe o mesmo ID de Região (*idRegiao*) do primeiro vencedor, mas que possua um ID diferente, incentivando o turismo regional.
- **Explore mais (Bubble Sort)**: O sistema lista outras opções no mesmo estado, organizadas alfabeticamente. O algoritmo opera em duas etapas: primeiro, coleta os IDs dos locais do estado em um vetor temporário (*vetorBubble*), aplicando uma regra de exclusão que impede que os locais já vencedores apareçam nesta lista secundária. Em seguida, executa o *Bubble Sort* (*loopExterno/loopInterno*), comparando byte-a-byte (*lb*) os valores ASCII das iniciais dos nomes para realizar a ordenação.

3 Arquitetura e Implementação da Interface *Web*

A solução foi estruturada para abstrair a complexidade do código *Assembly*, oferecendo uma experiência mais interativa. O sistema integra três componentes principais: o *Front-end Web* (Interface), o *Middleware* (mediador em *Node.js*) e o *Back-end* em *Assembly* MIPS.

3.1 *Front-end Web* (Interface Gráfica) - *index.html*

O arquivo `index.html` atua como a camada de apresentação. Ele substitui a interação baseada em terminal (*CLI*) do simulador *MARS* por um formulário interativo e responsivo.

3.1.1 Estrutura de Dados e Coleta

O formulário *HTML* coleta as cinco variáveis determinísticas para o algoritmo de recomendação. Os valores são mapeados internamente para inteiros, conforme esperado pelo código *Assembly*:

Tabela 3: Mapeamento de Entradas do Usuário

Parâmetro JSON	Pergunta na Interface	Valores Mapeados (MIPS)
`estado`	Escolha o Estado de Destino	PR (1), SC (2), RS (3)
`custo`	Qual seu perfil de orçamento?	Econômico (1), Conforto/Luxo (2)
`atividade`	O que você prefere fazer?	Natureza (1), Museu (2), Diversão (3), Praia (4)
`sensorial`	Perfil sensorial (Neurodivergência)?	Calmo (1), Agitado (2)
`seletividade`	Possui seletividade Alimentar?	Sim (1), Não (0)

3.1.2 Processamento *JavaScript*

Quando o usuário clica no botão "Gerar Roteiro Personalizado", a função assíncrona `gerarRoteiro()` é executada. Primeiramente, ocorre a Coleta de Dados: o *script* coleta os valores selecionados (1, 2, 3, etc.) do formulário e os armazena em um objeto *JavaScript*. Em seguida, ele chama o *Back-end* (*Middleware*), enviando esses dados via requisição `POST` para o *endpoint* `/roteiro` do *middleware*, que está em *Node.js*. O corpo dessa requisição é uma *string JSON* contendo os cinco parâmetros de entrada coletados. Por fim, o sistema procede à exibição do resultado: após receber a resposta do *middleware*, se for um sucesso, o resultado é exibido na área `#resultado-texto`. Caso haja um erro, a mensagem correspondente é exibida no mesmo local.

3.2 *Middleware* (Orquestrador de I/O) - *middleware.js*

O *middleware*, desenvolvido em *Node.js* com *Express*, é o componente crítico que viabiliza a comunicação entre o protocolo *HTTP* (*Web*) e o fluxo de entrada/saída padrão (*StdIO*) do simulador *MARS*.

Ao receber uma requisição na rota `/roteiro`, é instanciado um subprocesso isolado. A implementação utiliza *flags* avançadas da *JVM* para garantir *performance* e economia de recursos no ambiente de nuvem:

```
java -Dfile.encoding=UTF-8 -Xmx128m -XX:+TieredCompilation
-jar Mars4_5.jar nc backend.mar
```

As configurações adotadas foram:

- `-Dfile.encoding=UTF-8`: Garante a renderização correta de acentos na saída do terminal.

- `-Xmx128m`: Restringe o uso de memória *RAM* a 128MB, prevenindo falhas por excesso de consumo (*OOM Kills*) no *container Docker*.
- `-XX:+TieredCompilation`: Habilita a compilação *JIT* (*Just-In-Time*) em camadas. Em conjunto com `TieredStopAtLevel=1`, instrui a *JVM* a priorizar o tempo de inicialização rápido (usando apenas o compilador C1) em vez de otimizações de longo prazo, o que é ideal para comandos de execução instantânea.
- `nc` (***No Copyright***): Argumento do *MARS* que suprime o cabeçalho de inicialização, limpando o fluxo de saída.

3.2.1 *Pipeline* de Dados e *Parsing*

O fluxo de dados segue três etapas rigorosas:

1. **Injeção de Entrada (*Input Stream*):** O *middleware* converte o *JSON* recebido em uma *string* formatada com quebras de linha (`\n`) e a escreve diretamente no `stdin` do processo *Java*, simulando a digitação humana instantânea.
2. **Captura de Saída (*Output Stream*):** O *middleware* acumula todos os dados emitidos pelo *MARS* através do evento `stdout.on('data')`.
3. **Higienização e Resposta:** O sistema intercepta o texto completo, descarta os *logs* de sistema anteriores à *tag* `VENCEDOR:` e retorna ao *frontend* apenas a recomendação turística limpa e formatada.

3.3 Infraestrutura, Containerização e *Deploy*

Para garantir que o sistema estivesse acessível publicamente e independente do ambiente de desenvolvimento local, foi adotada uma estratégia de *deploy* baseada em *containers*. A aplicação está atualmente hospedada e ativa no endereço: `https://srti.onrender.com`.

O principal desafio de infraestrutura foi a natureza híbrida da aplicação: o *middleware* requer o *runtime* do *Node.js*, enquanto o simulador *MARS* exige a *Java Virtual Machine* (*JVM*). Ambientes de hospedagem padrão geralmente suportam apenas uma linguagem por vez.

Para resolver isso, foi criado um *Dockerfile* personalizado que constrói uma imagem híbrida, garantindo que o *container* seja autossuficiente e contenha todas as bibliotecas necessárias para rodar o projeto. O processo de *build* segue as etapas:

1. **Imagem Base:** Utiliza-se a imagem oficial `node:18-slim` (baseada em *Debian*) para garantir um ambiente *Node.js* leve e otimizado.

2. **Injeção de Dependências:** Executa-se o comando `apt-get install` para instalar manualmente o *OpenJRE* (*Java Runtime Environment*) dentro do *container Linux*. Isso permite que o comando `java -jar` seja reconhecido pelo sistema operacional.

3. **Gerenciamento de Arquivos:** Copia-se o executável `Mars4_5.jar`, o código *Assembly* (`backend.mar`) e os arquivos do *frontend* para o diretório de trabalho.

O *deploy* foi realizado na plataforma de nuvem *Render* (*Platform as a Service - PaaS*). A escolha justificou-se principalmente pelo suporte nativo ao *Docker*, visto que a plataforma constrói a imagem automaticamente a partir do *Dockerfile* presente no repositório, sem a necessidade de configuração extra.

4 Resultados

A validação funcional do sistema de recomendação turística foi realizada através da execução do código Assembly MIPS no ambiente de simulação MARS (*MIPS Assembler and Runtime Simulator*). A metodologia de testes consistiu na validação de conjuntos predeterminados de entradas, simulando as escolhas de diferentes perfis de usuários, seguida da análise detalhada da saída gerada no console (via *syscall 4* e *1*).

Os cenários de teste foram pensados para verificar a funcionalidade das seguintes características do sistema:

- **Precisão do algoritmo de Score:** verificação da soma correta dos pesos (sensorial, atividade, custo) e aplicação da penalidade de segurança (-100 pontos).

- **Gestão de múltiplos vencedores:** validação do uso do *buffer* de memória para armazenar e exibir múltiplos locais em caso de empate técnico.

- **Otimização de interface:** Confirmação de que o sistema ignora dados logísticos repetidos quando múltiplos vencedores pertencem à mesma cidade.

- **Ordenação e filtragem:** verificação da lista "explore mais", assegurando que a ordenação *Bubble Sort* está correta e que os locais vencedores foram devidamente excluídos desta lista secundária.

A seguir, são apresentados três testes que demonstram o comportamento do código diante de diferentes cenários.

4.1 Teste 1: Empate técnico complexo e otimização de interface

Neste cenário, exploramos a situação de equivalência de *score* do algoritmo: um empate quádruplo no estado de Santa Catarina. O objetivo é validar se o sistema consegue diferenciar logísticas de cidades distintas (imprimindo-as individualmente) e identificar logísticas idênticas na mesma cidade (suprimindo a repetição).

Simulamos um usuário buscando "museus"em Santa Catarina com perfil "calmo". Como a base de dados não possui museus cadastrados neste estado (atividade Match = 0), todos os locais calmos de SC empatam com a pontuação base (Base + Sensorial = 70 pontos).

Tabela 4: Entradas do Teste 1 (SC - Empate Múltiplo)

Parâmetro	**Opção Escolhida**	**Valor MIPS**
Estado	Santa Catarina	2
Custo	Econômico	1
Atividade	Museu	2
Perfil Sensorial	Calmo	1
Seletividade	Não	0

Análise do resultado:
O sistema identificou 4 locais vencedores com o mesmo Score (70).

- *Unipraias* (Balneário), *Zoo Pomerode* (Pomerode) e *Praia do Rosa* (Imbituba) estão em cidades diferentes. O sistema imprimiu corretamente a logística específica de cada um.

- *Oceanic Aquarium* também venceu, mas situa-se em Balneário Camboriú, compartilhando a infraestrutura do *Unipraias* (o primeiro vencedor). O algoritmo `verificaDuplicata` detectou que o endereço de memória do hotel era idêntico e suprimiu a repetição, exibindo a mensagem de otimização "(Mesma logística...)".

```
VENCEDOR (OPÇÃO RECOMENDADA):
Nome: Unipraias (Balneário - SC)
Score: 70
Hotel: Ibis Styles Balneário
Restaurante: Pizza Bis

----------------------------------------

ESSE LUGAR TAMBÉM ATINGIU O SCORE MÁXIMO!:
Nome: Zoo Pomerode (Pomerode - SC)
Score: 70
Hotel: Ap. Hardt
Restaurante: MAASS

----------------------------------------

ESSE LUGAR TAMBÉM ATINGIU O SCORE MÁXIMO!:
Nome: Praia do Rosa (Imbituba - SC)
Score: 70
Hotel: Pousada Rosa
Restaurante: Rest. Pousada

----------------------------------------

ESSE LUGAR TAMBÉM ATINGIU O SCORE MÁXIMO!:
Nome: Oceanic Aquarium (Balneário - SC)
Score: 70
(Mesma logística de Hotel/Restaurante sugerida acima)

[BATE-VOLTA]: Beto Carrero (Penha - SC)

[EXPLORE MAIS]:
- Beto Carrero (Penha - SC)
```

Para validar a integração completa do sistema, a saída do processador MIPS foi enviada para a Interface Web desenvolvida. As figuras a seguir demonstram a renderização final deste cenário de empate complexo. É possível observar visualmente a listagem dos quatro vencedores, a aplicação correta da mensagem de otimização logística ('Mesma logística...') para o *Oceanic Aquarium* e a formatação das seções de 'Bate-Volta' e 'Explore Mais', con-

firmando a experiência de usuário pretendida.

Figura 1: Interface Web exibindo o resultado do Teste 1: empate técnico entre 4 locais e supressão de logística repetida.

4.2 Teste 2: Empate em cidades diferentes

Neste cenário, o objetivo foi verificar se o sistema mantém a exibição completa das informações logísticas quando ocorre um empate entre locais de cidades distintas. Diferente do caso anterior, aqui é crucial que o usuário receba as recomendações de hotel específicas para cada destino.

Simulamos uma viagem para Santa Catarina (SC) com perfil econômico, focado em natureza e ambiente calmo.

Tabela 5: Entradas do Teste 2 (SC - Calmo)

Parâmetro	**Opção Escolhida**	**Valor MIPS**
Estado	Santa Catarina	2
Custo	Econômico	1
Atividade	Natureza	1
Perfil Sensorial	Calmo	1
Seletividade	Não	0

Resultado obtido: Houve empate entre *Unipraias* (Balneário Camboriú) e *Zoo Pomerode* (Pomerode), ambos com score máximo. Como os ponteiros de hotel apontam para endereços de memória diferentes, a rotina

`verificaDuplicata` permitiu a impressão completa das sugestões para ambos os vencedores, garantindo que o usuário tenha as informações de hospedagem corretas para cada cidade.

```
VENCEDOR (OPÇÃO RECOMENDADA):
Nome: Unipraias (Balneário - SC)
Score: 100
Hotel: Ibis Styles Balneário
Restaurante: Pizza Bis

----------------------------------------

ESSE LUGAR TAMBÉM ATINGIU O SCORE MÁXIMO!:
Nome: Zoo Pomerode (Pomerode - SC)
Score: 100
Hotel: Ap. Hardt
Restaurante: MAASS
[BATE-VOLTA]: Beto Carrero (Penha - SC)

[EXPLORE MAIS]:
- Beto Carrero (Penha - SC)
- Oceanic Aquarium (Balneário - SC)
- Praia do Rosa (Imbituba - SC)
```

Para corroborar a execução correta da lógica no ambiente final, a figura 2 apresenta a renderização da resposta na Interface Web. A imagem confirma que o sistema é capaz de exibir múltiplos cards de recomendação completos quando a situação geográfica exige, garantindo que o usuário tenha todas as informações necessárias para decidir entre cidades diferentes.

Figura 2: Interface Web exibindo o resultado do teste 2: empate entre cidades diferentes, mostrando logística completa para ambos os vencedores.

4.3 Teste 3: Penalidade de segurança e seletividade alimentar

Este é o teste mais crítico do sistema, pois valida a lógica de proteção ao usuário autista. O objetivo é confirmar se o algoritmo elimina locais sensorialmente inadequados (penalidade de -100 pontos) e se respeita a restrição alimentar do usuário, ignorando a preferência de custo se necessário.

Simulamos um usuário em Santa Catarina que busca "diversão"e tem orçamento de "luxo", mas necessita de um ambiente "calmo"e possui "sele-

tividade alimentar".

Tabela 6: Entradas do Teste 3 (SC - Inclusão Crítica)

Parâmetro	**Opção Escolhida**	**Valor MIPS**
Estado	Santa Catarina	2
Custo	Conforto/Luxo	2
Atividade	Diversão	3
Perfil Sensorial	**Calmo**	**1**
Seletividade	**Sim**	**1**

Análise do algoritmo:

- O local *Beto Carrero* é classificado como "diversão"(Match), mas possui perfil "agito". Como o usuário solicitou "calmo", o algoritmo aplicou a penalidade de -100 pontos, reduzindo o score final para -30 e eliminando-o da disputa.
- O local *Oceanic Aquarium* é "diversão"e "calmo". Ele recebeu a pontuação máxima (100).
- Quanto à alimentação: o usuário pediu orçamento "luxo"(2), mas marcou seletividade "sim"(1). A lógica de segurança alimentar sobrescreveu a escolha de custo e forçou a sugestão do restaurante simples.

Resultado obtido:
O sistema recomendou corretamente o aquário, garantindo a segurança sensorial. A sugestão de restaurante foi "Pizza Bis"(simples/conhecido) em vez de "Pharol Gourmet"(sofisticado), validando a regra de seletividade.

```
VENCEDOR (OPÇÃO RECOMENDADA):
Nome: Oceanic Aquarium (Balneário - SC)
Score: 100
Hotel: Marambaia Hotel
Restaurante: Pizza Bis

[BATE-VOLTA]: Beto Carrero (Penha - SC)

[EXPLORE MAIS]:
- Beto Carrero (Penha - SC)
- Praia do Rosa (Imbituba - SC)
- Unipraias (Balneário - SC)
- Zoo Pomerode (Pomerode - SC)
```

A figura 3 ilustra o resultado final na interface do usuário, confirmando visualmente a eficácia das regras de inclusão implementadas no código Assembly.

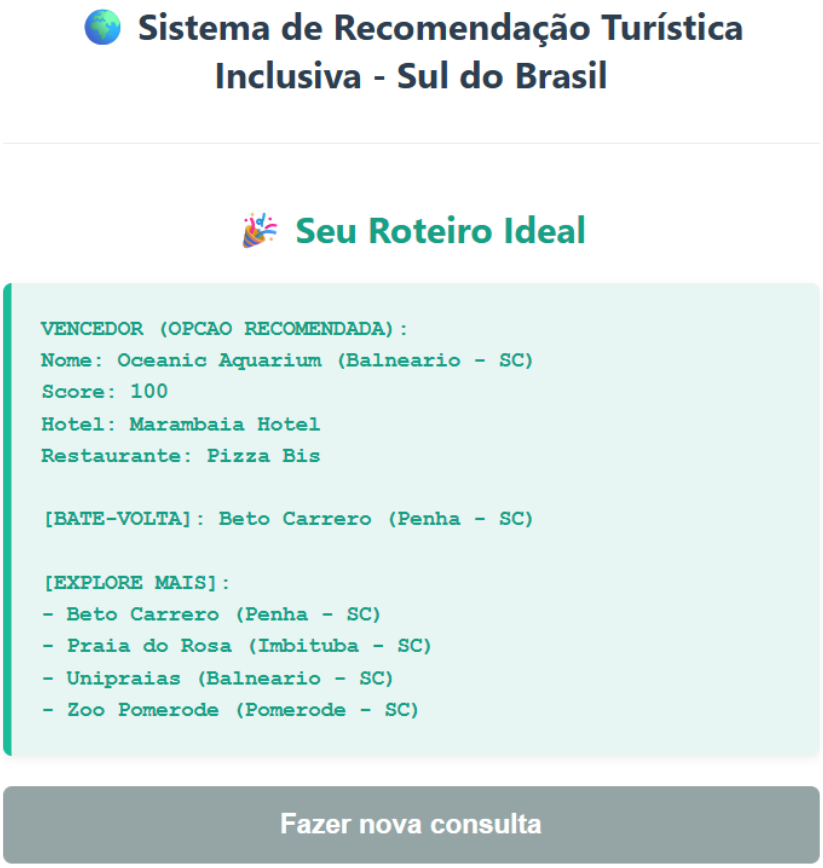

Figura 3: Interface Web exibindo o resultado do Teste 3: vencedor adequado ao perfil sensorial e restrição de restaurante simples devido à seletividade.

5 Planejamento e cronograma de execução

Cronograma de execução

Atividade	Integrantes	Meses				
		Out/1	**Out/2**	**Nov/1**	**Nov/2**	**Dez/1**
1. Escrita relatório (entrega 1)	Caio Peternela João Mancio José Luiz Natália Vitória					
2. Pesquisa sobre possíveis referências (entrega 1)	Caio Peternela João Mancio José Luís Natália Vitória					
3. Definição e busca dos destinos	Caio Peternela João Mancio José Luís Natália Vitória					
4. Mapeamento da memória RAM (.data) e implementação da arquitetura de vetores	João Mancio					
5. Desenvolvimento das sub-rotinas de I/O com validação de pilha e blindagem de inputs	João Mancio					
6. Codificação da lógica condicional para seleção de logística (Custo/Seletividade) e busca por região	João Mancio					
7. Implementação do cálculo de Score Ponderado e lógica de penalidade sensorial	Natália Vitória					
8.Desenvolvimento da estrutura de Buffer dinâmico para gerenciamento de múltiplos vencedores	Natália Vitória					
9. Implementação do algoritmo Bubble Sort e rotinas de otimização de saída (filtro de duplicidade)	Natália Vitória					
10. Arquitetura da Solução e Prova de Conceito (PoC)	Caio Peternela					
11. Implementação do Middleware	Caio Peternela					
12. Interface gráfica do usuário através do arquivo index.html	José Luiz					
13. Escrita relatório (entrega 2)	José Luiz					

Tabela 7: Cronograma do Projeto

A fase inicial do projeto (Atividade 1) consistiu na elaboração do primeiro relatório. Todos os integrantes participaram ativamente da escrita, definindo o escopo do sistema, os objetivos e a metodologia de desenvolvimento. Esta etapa foi fundamental para alinhar a visão do grupo e estabelecer os requisitos para a implementação técnica posterior.

Paralelamente, o grupo dedicou-se à pesquisa de referências bibliográficas (Atividade 2) para embasar tecnicamente o projeto. Todos os membros buscaram estudos sobre acessibilidade em turismo e as especificidades do TEA. Essa pesquisa permitiu identificar as melhores práticas para a construção

da interface web e para a lógica de pontuação sensorial, garantindo que o software atendesse às necessidades reais do público-alvo.

Caio, João, José e Natália foram responsáveis pela busca dos locais recomendados pelo sistema (Atividade 3). Eles realizaram a coleta, seleção e organização das informações referentes aos principais destinos turísticos da região Sul, abrangendo categorias como restaurantes, hotéis, praias e parques de diversões. Além disso, estruturaram os dados de forma padronizada, garantindo a consistência para o funcionamento do sistema de recomendação.

João Mancio ficou responsável pela estruturação da memória e pelo controle de entrada e saída de dados no código Assembly. Sua atuação incluiu o mapeamento da seção `.data`, onde implementou a organização dos vetores paralelos e a técnica de reutilização de endereços de memória para otimizar o armazenamento das strings de logística. Ele também desenvolveu as sub-rotinas de interação com o usuário, aplicando a manipulação do *Stack Pointer* (`$sp`) para validar se os números inseridos estavam dentro dos limites permitidos. Além disso, codificou as estruturas condicionais que selecionam as sugestões de hospedagem e alimentação baseando-se nas restrições orçamentárias e alimentares definidas nas entradas, bem como a busca linear para a funcionalidade de "Bate-Volta".

Natália Vitória desenvolveu os algoritmos de cálculo e ordenação do sistema. Ela implementou a lógica aritmética que define a pontuação (*Score*) de cada local, incluindo a subtração de pontos para casos de incompatibilidade sensorial. Para solucionar a questão de empates técnicos, criou a estrutura de *buffer* na memória, permitindo o armazenamento e exibição de múltiplos locais vencedores simultaneamente. Sua contribuição também abrangeu a codificação do algoritmo *Bubble Sort* para organizar alfabeticamente a lista de sugestões adicionais e a criação da rotina de comparação de endereços, que impede a impressão duplicada de informações logísticas na tela final.

Caio Peternela ficou responsável pela (Atividade 06), na qual desenvolveu um *Middleware* completo em *Node.js* para realizar a comunicação e a integração entre o Frontend e o simulador MARS. Ele estruturou o recebimento dos dados enviados pelo Frontend, preparou o envio dessas informações para o simulador, tratou as respostas geradas e padronizou o retorno para o restante da aplicação. Além disso, foi responsável pela realização da Prova de Conceito (*PoC*), que validou tecnicamente a comunicação via pipes, verificando que o *MARS* aceitava entrada direta pelo terminal e garantindo que a integração pudesse ser feita sem qualquer modificação no código *Assembly* original. Essa etapa permitiu confirmar a viabilidade da solução e orientar a implementação final.

A implementação do *Middleware* (atividade 7) também foi de responsabilidade do Caio. Ele implementou toda a camada *Middleware* da aplicação,

construindo a *API* em *Node.js* utilizando o *framework Express*. Ele definiu o endpoint responsável por receber os dados enviados pelo *Frontend*, configurou o processamento de requisições em formato *JSON* e estruturou a extração organizada dos parâmetros necessários para a execução do simulador. Também desenvolveu a lógica que cria uma nova instância do *MARS* a cada requisição, garantindo que cada usuário tenha um processo independente e evitando conflitos de memória entre execuções simultâneas.

Além disso, ele estruturou todo o fluxo de comunicação entre o *Node.js* e o simulador, implementando o envio automático das respostas do usuário para dentro do processo Java e configurando a captura contínua da saída produzida pelo *MARS*. Esse processamento envolveu reunir as informações geradas, limpar mensagens irrelevantes e converter tudo em um formato estruturado para ser retornado ao *Frontend*. Por fim, elaborou toda a lógica de finalização da execução e padronização das respostas enviadas ao cliente, consolidando assim a integração entre o sistema web e o simulador *Assembly*.

Já José Luiz ficou responsável pela criação da interface do usuário *web* através do arquivo *index.html* (atividade 8), substituindo a interação tradicional via console do *MIPS* por uma interface *web* acessível e alinhada ao objetivo do projeto de democratizar o lazer para pessoas autistas.

Também criou o formulário responsável por coletar os cinco critérios de recomendação turística — estado de destino, perfil de custo, atividade preferida, preferência sensorial e restrição alimentar —, estruturando a entrada dos dados de maneira clara e objetiva. Além disso, implementou a lógica em *JavaScript* para capturar as informações do formulário, transformá-las em um objeto *JSON* e enviá-las ao servidor por meio da *Fetch API*.

Por fim, desenvolveu o módulo de apresentação dos resultados, mantendo a formatação original das respostas do *MIPS*. Além disso, José também ficou responsável pela (atividade 9), elaboração do relatório final. Portanto, reuniu, organizou e integrou a parte de cada membro no documento para o envio final.

6 Conclusão

Este trabalho teve como objetivo aplicar os conceitos de arquitetura e organização de computadores no desenvolvimento de um sistema interativo em *Assembly MIPS* para o simulador *MARS*, que atua como um guia turístico personalizado.

O objetivo geral foi cumprido através da divisão do sistema em três componentes principais. O *Front-end Web*(*index.html*) foi desenvolvido como interface gráfica para coletar as respostas do quiz de forma amigável. O

Middleware em *Node.js* foi implementado para funcionar como orquestrador, recebendo os dados do *front-end* e canalizando-os para a entrada padrão do MARS. O *Back-end Assembly MIPS* foi configurado como o componente central, executando a lógica central do sistema a partir dos dados fornecidos.

No *MIPS*, foram aplicados os conceitos fundamentais solicitados: o controle de fluxo para gerenciar a sequência do quiz e a geração do roteiro; a manipulação de memória para armazenar e acessar as respostas, perfis e o catálogo de locais; o uso de sub-rotinas para organizar o código em funções reutilizáveis; e as operações de entrada/saída para receber os dados do *middleware* e exibir os resultados.

Dessa forma, o sistema desenvolvido processa as respostas do quiz interativo, calcula um score personalizado e gera um roteiro de visitação adequado ao perfil do usuário, incluindo atividades ao redor do destino principal. O projeto demonstrou, na prática, a construção de um sistema funcional e organizado em MIPS, integrado a uma interface *web* moderna, conforme os requisitos estabelecidos.

Apesar de ser funcional, a principal limitação do código em *Assembly MIPS* reside em sua baixa escalabilidade e dinamismo. A base de dados dos locais turísticos, por estar estaticamente codificada na memória *RAM* simulada (seção *.data*), impede que o sistema seja atualizado, pois a inclusão de um novo local ou a modificação de um atributo de acessibilidade exigiria a recompilação manual de todo o código. Além disso, o algoritmo de recomendação é inerentemente simplificado, não podendo processar a complexidade das variáveis ambientais em tempo real devido às restrições de recursos e *syscalls* do ambiente de simulação *MIPS*.

Ademais, na projeção de desenvolvimentos futuros, gostaríamos de expandir o escopo do projeto. O prazo limitado não permitiu a construção de um banco de dados maior para os destinos, nem a ampliação da cobertura para as outras regiões do Brasil.

Outra melhoria possível consiste no cálculo da distância da residência do usuário até o destino final. Isso permite que ele ja saiba a distância e coordenadas exatas do destino vencedor. Dessa forma, a sugestão de viagem poderia ser mais próxima de sua residência, se assim ele desejar, tornando o sistema mais viável economicamente.

Da mesma forma, o projeto ganharia valor significativo com a inclusão de imagens do destino vencedor, juntamente com o hotel e o restaurante selecionados. Essa funcionalidade permitiria que o usuário tivesse uma noção visual do local através do próprio sistema. Por fim, destaca-se a impossibilidade de implementar um sistema de agendamento de visitas e gestão de horários. Essa funcionalidade seria um recurso de acessibilidade valioso, permitindo que pessoas neurodivergentes evitassem filas e horários de pico, prevenindo

crises sensoriais, porém a limitação de recursos tornaram essa implementação inviável.

Referências

[1] Ministério do Turismo, "Pesquisa revela que 44% dos brasileiros nunca fizeram turismo no país," *Portal Governo do Brasil*, 2020. Disponível em: https://www.gov.br/turismo/pt-br/assuntos/noticias/pesquisa-revela-que-44-dos-brasileiros-nunca-fizeram-turismo/-no-pais.

[2] Agência Brasil, "Pesquisa mostra que brasileiro viajou pouco no 3º trimestre de 2019," *Agência Brasil*, 2020. Disponível em:https://agenciabrasil.ebc.com.br/economia/noticia/2020-08/pesquisa-mostra-que-brasileiro-viajou-pouco-no-3o-trimestre\-de-2019.

[3] M. L. A. Mora, "Turismo e acessibilidade: Análise dos obstáculos enfrentados por pessoas autistas durante a prática turística," *Monografia apresentada ao Centro de Excelência em Turismo da Universidade de Brasília*, 2022. Disponível em:https://bdm.unb.br/bitstream/10483/32961/1/2022_MariaLuizaAraujoMora_tcc.pdf.

[4] A. Leitão, "5 cidades em minas para um roteiro botânico incrível nesta primavera," *Estado de Minas*, 2025. Disponível em: https://www.em.com.br/turismo/2025/09/7255364-5-cidades-em-minas-para-um-roteiro-botanico-incrivel-nesta-primavera.html.

[5] G. L. Caetano and G. R. Pereira, "A acessibilidade e a inclusão de autistas em museus e centros de ciências: uma revisão sistemática," *Revista Ciências & Ideias ISSN: 2176-1477*, vol. 16, no. 1, p. e25162706, 2025. Disponível em: https://revistascientificas.ifrj.edu.br/index.php/reci/article/view/2706.

[6] Secretaria de Comunicação Social, "Governo do brasil lança nova linha de cuidados para tea e anuncia investimento anual de r$ 3,2 milhões para o paraná," *Portal Gov.br*, 2023. Disponível em: https://www.gov.br/secom/pt-br/assuntos/noticias-regionalizadas/saude-tea-e-pcds/governo-do-brasil-lanca-nova-linha-de-cuidados-para-tea-e-/anuncia-investimento-anual-de-r-3-2-milhoes-para-o-parana.

[7] Canal Autismo, "Autistas da região sul falam sobre clima, alimentação, inclusão e acesso a diagnóstico em podcast," *Portal Canal Autismo*, 2024. Disponível em: https://www.canalautismo.com.br/noticia/autistas-da-regiao-sul-falam-sobre-clima-alimentacao-inclusao/-e-acesso-a-diagnostico-em-podcast/.

[8] Secretaria da Saúde do Rio Grande do Sul, "Conheça o programa teacolhe," *Portal da Secretaria da Saúde do Rio Grande do Sul*, 2023. Disponível em: https://saude.rs.gov.br/conheca-o-programa-teacolhe.

[9] T. Grandin, *Pensando em imagens: minha vida com autismo*. Rio de Janeiro: Rocco, 2011.

[10] D. L. Fernandes and E. T. Bernier, "Acessibilidade virtual em destinos turísticos inteligentes: Os casos de curitiba [brasil] e málaga [espanha]," *Revista Rosa dos Ventos - Turismo e Hospitalidade*, vol. 13, no. 1, 2021.